

Tarea: Análisis probabilístico y algoritmos aleatorizados

Integrantes

- Christian Echeverría 221441
- Gustavo Cruz 22779
- Josue Say 22801
- Mathew Cordero 22982
- Pedro Guzmán 22111

Problema 1

Descripcion

Describe una implementación de un generador de números enteros aleatorios dentro del intervalo $([a, b])$, llamada `random(a, b)`. Su generador debe estar basado en llamadas a otro generador de bits aleatorios ($p(0) = p(1) = \frac{1}{2}$, donde (p) es la función de probabilidad) llamado `random(0, 1)`. Luego, calcule el tiempo de ejecución esperado de su algoritmo.

Hint: tome en cuenta que la probabilidad para cada número desde (a) hasta (b) debe ser la misma y no olvide que las probabilidades deben sumar 1. Investigue la distribución geométrica.

Solucion

La idea es contruir un generador de numeros aleatorios a partir de un generador binario, este generador hace numeros en una probabilidad de 50% que caiga 0 o 1.

Sabemos que b es el final de nuestro rango e incluimos a este y a a como parte de los numero aleatorios a generar. Por ende el numero de 0s o 1s que debe generar nuestro generador de numeros binarios debe ser del mismo rango que estos o un poco mas grande, pero nunca mas pequeño.

Un ejemplo es 6, 6 si lo tomamos como b el numero de binario para representarlo es 110. Lo puedes ver el numero es de 3 bits por lo que no debe de ser mayor a ese numero de rango de numeros generado por el generador binario.

Entonce si lo expresamos en una formula

$$k = n, 2^n \geq b$$

En una ecuacion seria de

$$k = \log_2(N)$$

Donde N es el tamaño del rango:

$$N = b - a + 1$$

Nuestro numero de digitos 0s o 1s sera k que es la potencia del numero mayor mas pequeño despues del b .

Entonces nuestro generador de numeros binarios debe de generar 3 caracteres, y verifica que el numero en decimal es una representacion valida dentro del rango de $[a, b]$ si esta fuera del rango recalcula.

La distribucion geometrica dicta lo siguiente

$$P(X = k) = (1 - p)^{k-1}p$$

- X es el numero de intentos
- p es la probabilidad de exito en un intento,
- $E[x] = \frac{1}{p}$

El random de generadores de a a la b seria que la probabilidad de exito seria

$$p = \frac{N}{2^k}$$

El numero esperado de intentos seria

$$E[X] = \frac{1}{p} = \frac{2^k}{N}$$

Ahora el tiempo de ejecucion seria

$$Tiempoesperado = k * \frac{2^k}{N}$$

Por lo que si lo ponemos en notacion asintotica nos da que

$$T = k \cdot \frac{2^k}{N} = O(k) \cdot \frac{2^k}{N}$$

El algoritmo seria el siguiente

Clase RandomImp:

Metodo random01Generator:

Retornar un número aleatorio entre 0 y 1 (inclusive)

Metodo random06Generator:

```

Inicializar val como N

Mientras val >= k hacer:
    res ← cadena vacía

    Repetir k veces:
        bit ← random01Generator()
        res ← res concatenado con bit como cadena

    bg ← convertir res de binario a entero
    val ← bg

Imprimir val

```

Ahora para la complejidad del tiempo tendríamos que

$$\frac{2^k}{N}$$

$$\frac{2^{\log_2(N)}}{N} = \frac{N}{N}$$

$$\frac{2^{\log_2(N)}}{N} = \frac{N}{N} = 1$$

Entonces

$$T = O(k * 1) = O(\log_2(N))$$

Donde $N = (b - a + 1)$

Problema 2

Descripcion

2. Dado un generador de bits llamado **biased-random()** que produce 1 con probabilidad (p) y 0 con probabilidad ($1 - p$) (donde ($0 < p < 1$)), diseñe un algoritmo que use **biased-random()** para generar 1 o 0, ambos con probabilidad ($\frac{1}{2}$). Calcule el tiempo de ejecución esperado.

Hint: ¿cuál es la probabilidad de que dos ejecuciones seguidas de **biased-random()** den el mismo resultado? Observe que las ejecuciones de **biased-random()** son independientes entre sí.

Solucion

Para este sabemos que

- La probabilidad de generar 1 es dada por p
- La probabilidad de generar 0 es dada por el complemento de 1. Osea que si da 80% el 1 , 0 sera de 20%.

Para analizar entonces el tiempo de espera sera de

Para obtener la probabilidad sera de la siguiente forma:

- Si obtenemos (0,0) o (1,1): descartamos y volvemos a intentar
- Si obtenemos (0,1): devolvemos 0
- Si obtenemos (1,0): devolvemos 1

Entonces tendremos que $P(0, 1) = p(1 - p)$ y $P(1, 0) = (1 - p)p$

$$P(0, 0) = (1 - p)(1 - p) = (1 - p)^2$$

$$P(0, 1) = (1 - p)(p) = p(1 - p)$$

$$P(1, 0) = (p)(1 - p) = p(1 - p)$$

$$P(1, 1) = (p)(p) = p^2$$

Por ende la probabilidad de que 2 tengan el mismo resultado es

$$P(\text{mismo resultado}) = P(1, 1) + P(0, 0) = p^2 + (1 - p)^2$$

La probabilidades de exito serian de

$$P(\text{exito}) = P(0, 1) + P(1, 0) = 2p(1 - p)$$

Y la probabilidad de fracaso es de

$$P(\text{fracaso}) = P(0, 0) + P(1, 1) = (1 - p)^2 + p^2 = 1 - 2p(1 - p)$$

Como X es parte de una distribucion geometrica entonces

$$E[X] = \frac{1}{P(\text{exito})} = \frac{1}{(2p(1 - p))}$$

Por lo que tomarías en tiempo de ejecución en

$$T(p) = O\left(\frac{1}{2p(1-p)}\right)$$

$$T(p) = O\left(\frac{1}{p(1-p)}\right)$$

Problema 3

Descripción

¿Cuáles son las probabilidades de obtener el **best-case scenario**, contratar sólo una vez, y el **worst-case scenario**, contratar n veces, si la distribución del input del *hiring problem* es aleatoria uniforme?

Solución

Sea n el número total de candidatos y sea C la variable aleatoria que cuenta cuántas veces se hace una contratación.

1. **Best-case scenario** ($C = 1$)

Sólo se contrata una vez cuando el **mejor** candidato de los n aparece en la primera posición.

Como todas las permutaciones de los n candidatos son igualmente probables, la probabilidad de que el mejor este en la posición 1 es

$$P(C = 1) = P(\text{mejor candidato en la posición 1}) = \frac{1}{n}.$$

2. **Worst-case scenario** ($C = n$)

Se contrata a todos los candidatos exactamente cuando cada nuevo candidato supera en mérito a todos los anteriores, es decir, la secuencia de méritos está estrictamente ordenada de menor a mayor. En las $n!$ permutaciones posibles, sólo una es estrictamente creciente:

$$P(C = n) = P(\text{orden estrictamente creciente}) = \frac{1}{n!}.$$

Para resumir, las respuestas son:

$$P(\text{best-case}, C = 1) = \frac{1}{n},$$

$$P(\text{worst-case}, C = n) = \frac{1}{n!}.$$

Problema 4

Descripción

Use variables aleatorias para calcular la suma esperada de n dados lanzados a la vez.

Solución

1. Definimos las variables aleatorias:

- Sea X_i la variable aleatoria que representa el valor del dado i , para $i = 1, 2, \dots, n$.
- Cada dado tiene un valor esperado:

$$\mathbb{E}[X_i] = \frac{1 + 2 + 3 + 4 + 5 + 6}{6} = \frac{21}{6} = 3.5$$

2. La suma total de los dados es:

$$S = \sum_{i=1}^n X_i$$

3. Usamos linealidad de la esperanza:

$$\mathbb{E}[S] = \sum_{i=1}^n \mathbb{E}[X_i] = n \times 3.5 = 3.5n$$

La suma esperada es $3.5 \times n$.

Problema 5

Descripción

Suponga una lista A con una permutación aleatoria de los números $[1..n]$. Una **inversión** es una pareja (i, j) donde $i < j$ pero $A[i] > A[j]$. Use variables aleatorias indicadoras para calcular el número esperado de inversiones en A .

Hint: ¿cuál es la probabilidad de que un par de posiciones i, j cualesquiera en una permutación aleatoria de n números cumplan con que $A[i] > A[j]$?

Solución

1. Definimos Variables Aleatorias Indicadoras:

- Sea $X_{i,j}$ una variable indicadora que vale:

$$X_{i,j} = \begin{cases} 1 & \text{si } A[i] > A[j] \text{ y } i < j, \\ 0 & \text{en otro caso.} \end{cases}$$

2. Queremos calcular:

$$\mathbb{E} \left[\sum_{i < j} X_{i,j} \right]$$

Por linealidad de la esperanza:

$$\mathbb{E} \left[\sum_{i < j} X_{i,j} \right] = \sum_{i < j} \mathbb{E}[X_{i,j}]$$

3. Calculamos $\mathbb{E}[X_{i,j}]$:

- En una permutación aleatoria, para cualquier par (i, j) con $i < j$, $\Pr(A[i] > A[j]) = \frac{1}{2}$, porque ambos órdenes son igualmente probables.
- Entonces: $\mathbb{E}[X_{i,j}] = \frac{1}{2}$.

4. Calculamos la Suma:

- El número total de pares (i, j) con $i < j$ es $\binom{n}{2} = \frac{n(n-1)}{2}$.
- Por lo tanto, el valor esperado es:

$$\mathbb{E}[\text{Número de inversiones}] = \frac{1}{2} \cdot \binom{n}{2} = \frac{1}{2} \cdot \frac{n(n-1)}{2} = \frac{n(n-1)}{4}$$

El número esperado de inversiones en A es $\frac{n(n-1)}{4}$.

Problema 6

a. Explicación de la fórmula usada para calcular $P[\text{Ik}]$ en la prueba del teorema 2

En la prueba del teorema 2, la fórmula usada para calcular $P[\text{Ik}]$ es:

$$P[I_k] = \frac{\binom{n}{k} \cdot (n-k)!}{n!}$$

Esta fórmula calcula la probabilidad de que al menos los primeros k elementos de una lista esten en orden. El razonamiento detrás de esta fórmula es el siguiente:

- El numerador representa el producto de: el número de posibilidades para elegir k primeros elementos que esten en orden correcto, multiplicado por el número de posibilidades para organizar los $n-k$ elementos restantes al final del arreglo.
- El denominador es simplemente el número total de arreglos posibles de longitud n .

Al simplificar esta fracción, obtenemos:

$$P[I_k] = \frac{1}{k!}$$

Esta fórmula es crucial porque establece la relación entre I_k (la variable aleatoria que indica si al menos los primeros k elementos están en orden) y la variable C que cuenta el número de comparaciones realizadas por el algoritmo.

b. Por que $E[C] = \sum_{k>0} P[I_k]$ en la prueba del teorema 2

En la prueba del teorema 2, se establece que $I_k = 1 \Leftrightarrow C \geq k$. Esto significa que la variable aleatoria I_k (que indica si los primeros k elementos están en orden) es igual a 1 si y solo si el número de comparaciones C es mayor o igual a k .

Por lo tanto: - $P[C \geq k] = P[I_k]$

Usando una propiedad fundamental de las variables aleatorias no negativas, sabemos que: - $E[C] = \sum_{k>0} P[C \geq k]$

Combinando estas dos ecuaciones: - $E[C] = \sum_{k>0} P[I_k]$

Esta fórmula permite calcular el valor esperado de C sumando las probabilidades de que al menos los primeros k elementos esten ordenados, para todos los valores posibles de k . Al sustituir $P[I_k] = 1/k!$, llegamos a:

$$E[C] = \sum_{k=1}^{n-1} \frac{1}{k!} \sim e - 1$$

c. Por que la variable aleatoria I tiene distribución geometrica en la sección 2.3

La variable aleatoria I , que cuenta el número de iteraciones en bogo-sort, tiene una distribución geometrica por las siguientes razones:

1. **Proceso de Bernoulli:** En cada iteración, el algoritmo genera una permutación uniforme al azar del arreglo.
2. **Criterio de detención:** El algoritmo continúa iterando hasta encontrar la secuencia ordenada por primera vez.
3. **Probabilidad constante:** La secuencia ordenada se encuentra con probabilidad $1/n!$ en cada prueba.
4. **Independencia:** Las iteraciones son independientes entre sí (cada permutación es generada de manera completamente aleatoria).

Por lo tanto, la distribución de probabilidad de I es:

$$P[I = i] = \left(\frac{n! - 1}{n!} \right)^{i-1} \cdot \frac{1}{n!}$$

Esto corresponde exactamente a la definición de una variable aleatoria con distribución geométrica, donde: - $p = 1/n!$ es la probabilidad de éxito (encontrar la secuencia ordenada) - $E[I] = 1/p = n!$

La distribución geométrica modela el número de intentos necesarios hasta obtener el primer éxito en una secuencia de experimentos de Bernoulli independientes, lo que describe perfectamente el comportamiento del algoritmo bogo-sort.

Problema 7

Pregunta 7a: Comparación entre Guess-Sort y $Bozo - Sort_{opt}^+$

Diferencia clave: - $Bozo - Sort_{opt}^+$: - Solo intercambia pares de elementos adyacentes si forman una inversión - Realiza verificaciones completas del orden del arreglo en cada iteración - Complejidad esperada: $O(n^2 \log n)$

- **Guess-Sort:**

- Compara pares de elementos **no adyacentes** seleccionados aleatoriamente
- Reduce la frecuencia de verificaciones completas (cada $n-1$ operaciones)
- Optimiza la detección de inversiones en arreglos parcialmente ordenados
- Misma complejidad teórica ($O(n^2 \log n)$) pero con mejor rendimiento práctico

Mejora principal: Guess-Sort acelera el proceso al: 1. Explorar un espacio mayor de posibles inversiones 2. Minimizar las costosas verificaciones de orden completo 3. Aprovechar mejor el azar en arreglos desordenados

Pregunta 7b: Tiempo de ejecución esperado de Fun-Sort según Teorema 6

Teorema 6 establece que: Para $n = 2^k - 1$, la búsqueda binaria en un arreglo desordenado termina en promedio en la posición correcta que tendría si estuviera ordenado.

Implicaciones para el average-case: 1. Cuando se cumplen las condiciones del teorema: - Los elementos se insertan cerca de su posición correcta - Se reducen significativamente los swaps innecesarios

2. Tiempo esperado:

- **Mejor caso teórico:** $\Theta(n \log n)$
- **Average-case estimado:** Entre $\Theta(n \log n)$ y $\Theta(n^2 \log n)$
- **Nota:** El paper no demuestra formalmente este límite inferior

Conclusión: El teorema sugiere que Fun-Sort podría alcanzar un tiempo sub-cuadrático en el average-case cuando $n = 2^k - 1$, pero esto queda como pregunta abierta en el paper.